Aula 4 — Maestro Mobile

Testes E2E em YAML, no celular de verdade (emulador)

Prof. Jackson Smith Moisés Matias
PUC Minas IEC · 15/10/2026

Recap da Aula 3

- **SPA e PWA:** o app só está pronto quando *sinaliza* que está pronto (nada de `sleep`)
- **Service Worker:** ciclo de vida, `controller`, cache e **offline**
- **Playwright:** `route()` para mockar rede, `setOffline()` para derrubar a internet
- **Lab Playwright PWA** (20 desafios) — termina em casa

> Hoje a ideia é a mesma, mas no **app mobile**: abrir, tocar, conferir — e sem `sleep`.

Agenda da aula (3h30)

- **45min — Setup guiado:** Android Studio, emulador, Maestro e o app CineFav rodando no *seu* computador
- **40min** — Maestro: o que é, anatomia de um flow, `id` (testID), Maestro Studio
- **30min** — Intervalo
- **95min — Lab Mobile** em sala: construímos os flows 01 a 03 juntos; em casa você termina o `02` e o `03`, mais o Jest (`04` e `05` são opcionais)

> Ninguém aqui precisa ter rodado um emulador antes. Esse é o motivo dos primeiros 45 minutos.

Objetivos da aula

- **Ter** um device Android (emulador) rodando o app, e o Maestro conversando com ele
- **Escrever** flows Maestro em YAML (`launchApp`, `tapOn`, `inputText`, `assertVisible`)
- **Usar** `id` (testID) em vez de texto para achar elementos — o flow não quebra se o texto mudar
- **Reduzir flakiness** mobile (animações, popups, tempo de tela)
- **Entregar** o Lab Mobile: testes Jest + flows Maestro `02` e `03` (`04` e `05` são opcionais)

Parte 1 — Setup

45 min · seu device rodando o CineFav

A meta: ver o CineFav aberto no emulador e o Maestro tocando nele

#	Passo	Tempo	Como saber que deu certo
1	Android Studio instalado	5 min	abre a tela "Welcome"
2	Emulador criado e ligado	10 min	você vê um celular Android na tela
3	`adb` no terminal	5 min	`adb devices` lista o emulador
4	Maestro instalado	5 min	`maestro --version` mostra um número
4b	**Maestro Studio** (app desktop)	5 min	o app abre e mostra o seu emulador
5	**App CineFav instalado, aberto e logado** no emulador	10 min	você vê a lista de filmes na tela do emulador
6	Primeiro flow verde	5 min	`maestro test flows/01-launch.yaml` passa

> **Travou em qualquer passo?** Levante a mão. Não espere chegar no fim para pedir ajuda.

Passo 1 — Android Studio

1. Baixe em **developer.android.com/studio** e instale (Next, Next, Finish)
2. Na primeira abertura: **Standard** → aceite as licenças → ele baixa o **SDK**, o `adb` e o **emulador** (alguns GB: deixe baixando e avance para o passo 4)
3. Você **não precisa** criar projeto nenhum. Só precisa do que ele instala

> 💡 **PC com pouca memória?** Pule o emulador e use o **celular Android** com cabo USB: *Configurações → Sobre o telefone → toque 7× em "Número da versão" → Opções do desenvolvedor → Depuração USB*.

Passo 2 — Criar o emulador (AVD)

Android Studio → ⋮ (More Actions) → Virtual Device Manager → Create Device

1. Escolha **Pixel 8** (ou outro Pixel)
2. Imagem do sistema: **API 34** ("UpsideDownCake") → baixe (**Download**) se estiver na lista
   - **Mac com chip M1/M2/M3:** imagem **arm64**. **Windows/Intel:** imagem **x86_64**
3. **Finish** → clique no ▶ ao lado do emulador e espere o Android ligar (1ª vez: 2–5 min)

> ⚠️ **Windows:** se der erro de virtualização, ligue a **virtualização (VT-x/SVM)** na BIOS e ative *Windows Hypervisor Platform*. Chame o professor.

Passo 3 — Fazer o terminal enxergar o `adb`

```
# macOS / Linux — acrescente no fim do ~/.zshrc (ou ~/.bashrc) e reabra o terminal:
export ANDROID_HOME=$HOME/Library/Android/sdk      # Linux: $HOME/Android/Sdk
export PATH=$PATH:$ANDROID_HOME/platform-tools:$ANDROID_HOME/emulator
```

Windows: *Editar as variáveis de ambiente do sistema* → crie **`ANDROID_HOME`** e acrescente `%ANDROID_HOME%\platform-tools` e `%ANDROID_HOME%\emulator` ao **Path**. **Reabra** o terminal.

> **Não chute o caminho:** Android Studio → *Settings* → *Android SDK* → copie o **Android SDK Location** (quem instalou em outra pasta tem outro caminho). Lá dentro tem que existir `platform-tools\adb.exe`. macOS/Linux: depois de editar, `source ~/.zshrc` (ou `~/.bashrc`) ou abra terminal novo.

```
adb devices        # com o emulador ligado: "emulator-5554   device"
```

> `adb: command not found`? O terminal não foi reaberto, ou o caminho do `ANDROID_HOME` está errado.

Passo 4 — Instalar o Maestro

O Maestro precisa do **Java 17 ou mais novo**. Confira: `java -version`
(sem Java: o **Android Studio já traz um** — ou instale o *Temurin 17*).

```
# macOS / Linux
curl -Ls https://get.maestro.mobile.dev | bash
source ~/.zshrc          # (ou ~/.bashrc, ou abra um terminal novo)
```

> **Windows?** Próximo slide (instalação por `.zip`, sem o one-liner).

Feche e **reabra o terminal**, depois: `maestro --version`

Passo 4 (Windows) — Maestro por `.zip`

Terminal nativo — PowerShell/CMD, não WSL.

1. Baixe o **`maestro.zip`** em *github.com/mobile-dev-inc/Maestro/releases/latest* e extraia em `C:\maestro`
2. Variáveis de ambiente → **Path** → acrescente **`C:\maestro\bin`** (tem que terminar em `\bin`)
3. **`JAVA_HOME`** = `C:\Program Files\Android\Android Studio\jbr` (o Java que o Android Studio já traz)
4. **Feche e reabra** o terminal (e o editor, se rodar o terminal dentro dele)

Confira: `maestro --version`

> Guia oficial: **docs.maestro.dev** → *Installing Maestro*. A página tem abas (*Studio · CLI · Cloud*): você quer a **CLI**. A documentação desencoraja o **WSL** no Windows: use o terminal nativo.

Passo 4b — Maestro Studio (app desktop)

Desde 2026 o Studio **não vem mais no `maestro`**: é um aplicativo à parte. Baixe e instale:

Sistema	Arquivo (em studio.maestro.dev)	Tamanho
macOS	`MaestroStudio.dmg`	~700 MB
Windows	`MaestroStudio.exe`	~560 MB
Linux	`MaestroStudio.AppImage`	~630 MB

1. Instale e abra o app
2. Com o **emulador ligado**, escolha o seu device na lista
3. Você deve ver o CineFav espelhado: clique num elemento e o Studio mostra o `id`

> ⚠️ **É um download grande:** o ideal é baixar **antes da aula**. Quem não conseguir hoje **não fica travado**: `maestro hierarchy` imprime os mesmos `id`s no terminal, e todos estão em `src/utils/testIDs.ts`.

Passo 5 — Instalar o CineFav e abrir no emulador

1 · Instalar (caminho rápido — APK pronto):

```
# baixe CineFav.apk em: github.com/jacksonsmith/puc-iec-testes-aplicacoes-mobile/releases
adb install CineFav.apk          # tem que terminar com: Success
```

2 · Abrir: no emulador, deslize a tela inicial para cima e toque no ícone **CineFav** 🎬.

3 · Conferir (é o checkpoint do setup):

- aparece a tela de **login** → entre com `aluno@puc.br` / `1234` → aparece a **lista de filmes** (Matrix, Avatar...)
- toque no ❤️ de um filme, abra a 🔍 busca e o botão de Favoritos

> ✅ **Só avance quando você estiver olhando a lista de filmes no emulador.** Se não abriu, levante a mão agora: o resto da aula depende disso.
> Mesmo app do lab, dados mockados (sem internet). O Maestro controla o app **já instalado**; ele não lê o `.apk`.

Passo 5b — Rodar do código (alternativa; em casa)

Mais demorado: a 1ª compilação baixa o Gradle (5–15 min). Com o emulador ligado:

```
cd exercicios/02-lab-mobile-maestro-jest/pratica
npm install
npm run android          # compila, instala e abre o CineFav no emulador
```

> Hoje usamos o APK para não gastar a aula esperando compilar. O resultado na tela é o mesmo.

Passo 6 — Prova de vida

```
cd exercicios/02-lab-mobile-maestro-jest/pratica
ls flows                                  # tem que listar 01-launch.yaml … (Windows: dir flows)
maestro test flows/01-launch.yaml         # o Maestro abre o app, faz login e confere a lista
```

✅ **Verde?** Você acabou de rodar um teste E2E. Tire um print e comemore.

Enquanto espera os colegas: reabra o app e repita o tour (login, lista, busca, favoritar, detalhe). Cada clique seu vira um comando do flow.

Travou? As causas mais comuns

Sintoma	O que fazer
`adb: command not found`	reabra o terminal; confira o `PATH` (passo 3)
`adb devices` vem **vazio**	o emulador terminou de ligar? Senão: `adb kill-server && adb start-server`
`adb devices` mostra **offline**	espere o boot terminar, ou reinicie o emulador
`maestro: command not found`	reabra o terminal; reinstale (passo 4)
Maestro reclama de **Java**	`java -version` precisa ser 17+; no Windows defina `JAVA_HOME` (passo 4) ou instale o Temurin 17
`Flow file not found`	você está na pasta errada: `cd` para `pratica/` (passo 6)
`driver did not start up in time` / `UNAVAILABLE`	o emulador está sobrecarregado: **rode de novo**; se repetir, `adb kill-server && adb start-server`
Emulador **lento/travando**	feche navegador e outros programas (o emulador usa 2–4 GB de RAM) ou use o celular com cabo USB
Jest avisa `worker process has failed to exit`	só um aviso: pode ignorar

Travou? (continuação) — Windows, emulador e APK

Sintoma	O que fazer
`maestro` não é reconhecido no **Windows**	o Path tem que apontar para `C:\maestro\bin` (com o `\bin`); reabra o terminal
Maestro diz **"no devices"** / não acha o device	o emulador precisa estar **ligado e com o Android carregado** antes do `maestro test`; confira com `adb devices`
Emulador com **tela preta**	aperte o botão de energia do emulador (painel lateral) ou reinicie-o
Aparece um **aviso cinza com muito texto** (compatibilidade) ao abrir o app	toque em **OK** e rode o flow de novo. Se atrapalhar o teste, mande um print no chat
`adb install CineFav.apk` → *not found*	o `.apk` tem que estar **na pasta onde você roda o comando** (`pratica/`); confira com `ls` / `dir` e o nome exato (`CineFav.apk`)
`npm run android` deu erro	**ignore**: com o APK instalado você não precisa dele (é só a alternativa do passo 5b)
`npm test` deu erro depois de `npm install`	leia o erro; se pedir atualização/versão, mande o print no chat — o Maestro **não** depende do Jest

Parte 2 — Maestro

40 min · o que é, como se lê, como se acha o `id`

Por que Maestro

1. **YAML declarativo** — você descreve *o que o usuário faz*, não como o código faz
2. **Espera embutida** — acabou o `sleep(3000)`: o Maestro repete até o elemento aparecer
3. **Setup em minutos** — um comando instala; sem servidor, sem driver
4. **Mesmo flow no iOS e no Android** — escreve uma vez
5. **Maestro Studio** — editor visual que *descobre os seletores por você*

> Quem testa mobile à mão hoje ganha aqui um jeito de **repetir** o roteiro sempre igual.

Quem usa Maestro em produção

Casos documentados:

Empresa	Contexto
Cash App (Block)	Reescreveu suíte iOS+Android completa
Reddit	Flows cross-platform únicos iOS+Android
Nubank	Mobile E2E de pagamentos (Brasil)
Grab	Super-app — 400+ flows em produção
Lingvano	React Native + Expo + Maestro — caso no Medium

Referências verificadas:

- Lingvano Eng. (2023) *"Native E2E testing with Expo, RN and Maestro"* — Medium
- LambdaTest (2024) *"Future of QA"* — survey 1.600+ QAs
- Linares-Vásquez et al. (2017) ICSME — base acadêmica automação mobile
- Wang et al. (2023) *"Empirical Study on Flaky Tests in Mobile Apps"* ASE 2023

IA + Maestro (2024–25):

- `maestro ai "descrição em PT"` → gera YAML automaticamente (beta)
- LLMs atualizam flows quando UI muda

Maestro — primeiro flow

```yaml
# login_flow.yaml
appId: com.myapp
---
- launchApp
- tapOn: "Email"
- inputText: "user@email.com"
- tapOn: "Senha"
- inputText: "secret"
- tapOn: "Entrar"
- assertVisible: "Bem-vindo"
```

```
maestro test login_flow.yaml
```

> Em 30 segundos você tem teste E2E rodando.

Maestro — comandos principais

```yaml
- launchApp                      # abre app
- tapOn: "Submit"                # tap por texto
- tapOn:
    id: "submit-button"          # tap por ID (preferido)
- inputText: "hello"             # digita
- swipe:
    start: "50%, 80%"
    end: "50%, 20%"
- scroll
- assertVisible: "Welcome"
- assertNotVisible: "Loading"
- pressKey: "BACK"
- waitForAnimationToEnd
- takeScreenshot: "after-login"
```

Maestro — conditional flows

```yaml
appId: com.myapp
---
- launchApp
- runFlow:
    when:
      visible: "Aceitar cookies"
    file: ./flows/dismiss-cookies.yaml
- runFlow:
    when:
      notVisible: "Welcome"
    file: ./flows/login.yaml
- assertVisible: "Welcome"
```

> Suíte se adapta ao estado do app. Sem `if` imperativo.

`tapOn: text` vs `tapOn: id` — use id sempre que possível

❌ **Frágil:**

```
- tapOn: "Cadastrar"
```

Quebra em: tradução, A/B test, refactor de texto.

✅ **Estável:**

```
- tapOn:
    id: "login-submit-button"
```

Independe de texto. App bem instrumentado expõe `testID` em todos os elementos críticos.

> **Regra:** `tapOn: id:` sempre que o app expõe testID. `tapOn: "texto"` só se não tiver ID.

Accessibility role — o mesmo elemento, duas lentes

```
- tapOn:
    id: "movie-card-heart-603"
```

Testa que o elemento existe. Mas o **leitor de tela** de um usuário cego não vê ID — vê:

```
accessibilityRole="button"
accessibilityLabel="Adicionar favorito"
```

> Se o componente não tiver esses props, o teste de ID passa mas a **acessibilidade real falha**. Testar por ID garante que o elemento **existe**; testar por role/label garante que é **usável**.

Maestro Studio — o que é

Ferramenta visual: você **clica no app** e ela escreve o YAML. Desde 2026 é um **aplicativo desktop separado** do CLI (baixe em **studio.maestro.dev**).

```
# abra o app "Maestro Studio" (Mac .dmg · Windows .exe · Linux .AppImage)
maestro hierarchy   # alternativa no terminal: imprime os elementos da tela com seus ids
```

O que você vê no Studio:

- Espelho ao vivo do simulador/device
- Clicou → gerou `tapOn:` automaticamente
- Digitou → gerou `inputText:`
- Inspetor de elementos (IDs, textos)
- Botão "Add assertion" → gera `assertVisible:`

Fluxo:

1. Simulador rodando com o app aberto
2. Abra o app **Maestro Studio** e escolha o device
3. Você interage no espelho do device
4. YAML vai sendo gerado ao lado
5. Salva → `maestro test flow.yaml`

> Não precisa saber YAML pra começar. Studio é o ponto de entrada.

Flakiness mobile — causas e mitigações

Causa	Mitigação Maestro
Animações	`waitForAnimationToEnd`
OS popups (permissões)	`runFlow when: visible:` para dispensar
Texto que muda (A/B test, tradução)	`tapOn: id:` em vez de texto
Biometria	Mock no build de test
Network jitter	Mock server local
Lifecycle (background→foreground)	Estabilizar antes de `assert`
Touch latency em emulator	Confirmar em device real se flaky

> **Wang et al. (2023)** ASE — estudo empírico sobre flaky tests em apps mobile. **iDFlakies (Lam et al. 2019)** catalogou causas — referências acadêmicas da área.

Pitfalls comuns

❌ **`tapOn:` por texto que muda** — flake em A/B test ou tradução; use `id:`
❌ **Não usar `runFlow` para login** — repetição em todos os testes
❌ **Suíte sem retry** — flake transitório vira falha permanente
❌ **Executar só em emulator** — esconde issues de touch latency
❌ **Não monitorar flake rate** — métrica essencial degrada silenciosamente

☕ Intervalo

30 minutos

*Na volta: o **Lab Mobile**. Deixe o **emulador ligado** e o terminal aberto em `pratica/`.*

Lab Mobile — o app: CineFav

App de filmes **já pronto** (React Native + Expo). **Você só escreve testes.**

Tela	O que tem	testID (exemplos)
Login	e-mail, senha, entrar	`login-email-input` · `login-submit-button`
Lista	filmes, busca, favoritos	`movielist-screen` · `movie-card-heart-603`
Detalhe	filme + favoritar	`detail-screen` · `detail-favorite-button`

- Login de teste: `aluno@puc.br` / `1234` · dados **mockados** (sem token, sem internet) → o teste é **determinístico**
- Lista completa de testIDs: `src/utils/testIDs.ts`

Parte B — Maestro (ao vivo, 60 min)

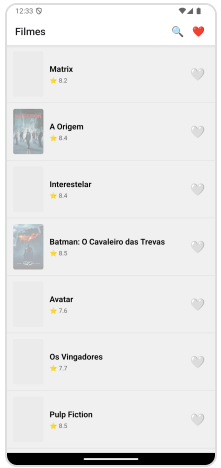
Flow	O que testa	Conceito
`01-launch`	login + lista	🟦 **modelo resolvido** — leia primeiro
`02-search`	busca	`env:` → `${SEARCH_TERM}` · **fazemos juntos**
`03-favorite`	favoritar → tela Favoritos	asserção em outra tela · **fazemos juntos**
`04-detail`	detalhe + favoritar	navegação · ⭐ opcional
`05-js-dynamic`	busca com termo sorteado	`evalScript` · ⭐ opcional

Travou no seletor? abra o **Maestro Studio** (ou rode `maestro hierarchy`): mostra o `id` de cada elemento.

Construindo `01-launch` — o modelo

Já resolvido — mas rodamos **ao vivo** primeiro, para ver o app abrir de verdade:

```
cd exercicios/02-lab-mobile-maestro-jest/pratica
maestro test flows/01-launch.yaml
```



Login (`aluno@puc.br` / `1234`) → a tela de login some e a lista de filmes aparece. Esse é o `assertVisible: id: "movielist-screen"` do flow passando.

Construindo `02-search` — juntos

O scaffold (`flows/02-search.yaml`) só tem os `TODO`:

```yaml
appId: com.puciec.cinefav
env:
  SEARCH_TERM: "Matrix"
---
- launchApp
# TODO: faça login (copie os passos do flow 01)
# TODO: abra a busca — id "movielist-search-button"
# TODO: toque em "search-input" e use inputText com ${SEARCH_TERM}
# TODO: assertVisible do ${SEARCH_TERM}
```

`env:` define uma variável com valor padrão; `${SEARCH_TERM}` lê o valor depois.

> ⚠️ **YAML indenta com ESPAÇOS, nunca com a tecla Tab.** O `.editorconfig` do projeto já converte para 2 espaços; sem ele, `tapOn:` seguido de `id:` precisa estar 2 espaços mais à direita.

Onde achamos os `id`s? Não é adivinhação — abrimos o **Maestro Studio** agora.

Achando o seletor com o Maestro Studio

```
# abra o app "Maestro Studio" (desktop) e escolha o emulador
maestro hierarchy  # sem o Studio: o mesmo, em texto, no terminal
```

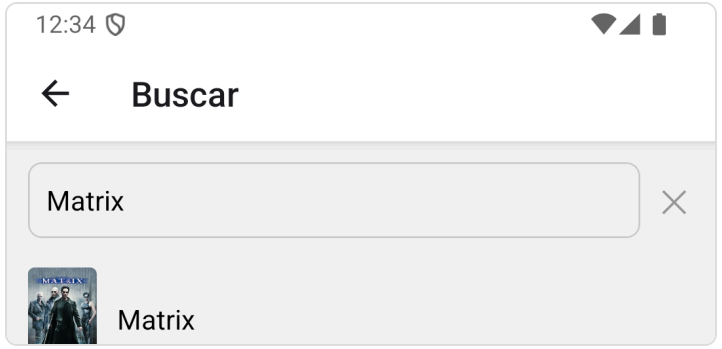
Com o app aberto na tela de busca: **botão direito** no campo → o menu mostra os comandos Maestro para aquele elemento, com `id: "search-input"`. No botão de busca do header → `id: "movielist-search-button"`.

Aponta e clica, não adivinha no código-fonte.

> **Não é copiar e colar cego.** O que sai do menu geralmente precisa de limpeza: prefira `tapOn: { id: ... }` a `tapOn: "texto"` e remova passo redundante.

`02-search` — completo e rodando

```yaml
- launchApp
- tapOn: { id: "login-email-input" }
- inputText: "aluno@puc.br"
- tapOn: { id: "login-password-input" }
- inputText: "1234"
- tapOn: { id: "login-submit-button" }
- tapOn: { id: "movielist-search-button" }
- tapOn: { id: "search-input" }
- inputText: ${SEARCH_TERM}
- assertVisible: ${SEARCH_TERM}
```

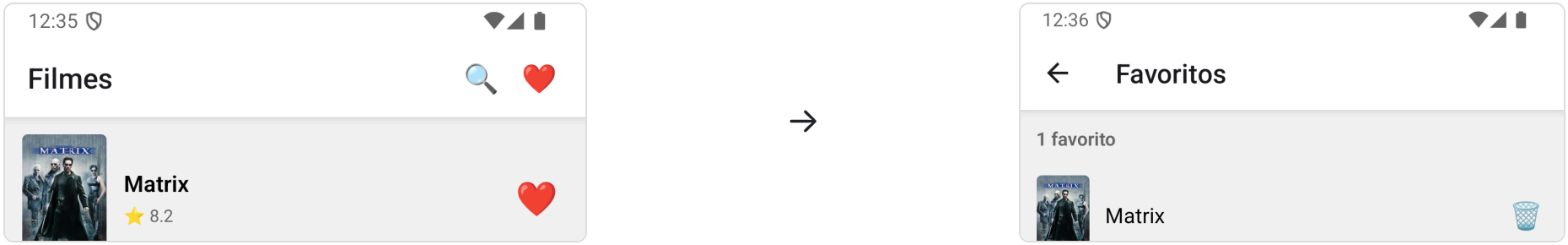


Troque o termo na hora, sem duplicar arquivo: `maestro test flows/02-search.yaml -e SEARCH_TERM=Avatar`

Construindo `03-favorite` — juntos

Conceito novo: **assert cross-tela** — a ação numa tela (favoritar na lista) aparece em outra (Favoritos) + contador.

```yaml
- launchApp
- tapOn: { id: "login-email-input" }
- inputText: "aluno@puc.br"
- tapOn: { id: "login-password-input" }
- inputText: "1234"
- tapOn: { id: "login-submit-button" }
- tapOn: { id: "movie-card-heart-603" }          # favorita Matrix na lista
- tapOn: { id: "movielist-favorites-button" }  # abre Favoritos
- assertVisible: { id: "favorites-item-603" }
- assertVisible: "1 favorito"
```



Reparou que repetimos o login 3 vezes? — Fragmento

Os flows `01`, `02` e `03` começam com os **mesmos 5 passos**. Extraia para um arquivo e reuse (**bônus** do lab):

```yaml
# flows/_fragments/login.yaml
appId: com.puciec.cinefav
---
- tapOn: { id: "login-email-input" }
- inputText: "aluno@puc.br"
- tapOn: { id: "login-password-input" }
- inputText: "1234"
- tapOn: { id: "login-submit-button" }
```

No flow: `- runFlow: file: _fragments/login.yaml`

> Se o login mudar (campo novo, botão em outro lugar), muda em **1 arquivo**, não em 5. É o mesmo "não repita conhecimento" de uma função auxiliar no código.

Prévia do `05-js-dynamic` (opcional, sem nota)

`evalScript` roda JavaScript dentro do flow; o que você grava em `output` é lido com `${output.campo}`:

```
- evalScript: ${output.termo = ['Matrix','Avatar','Interestelar'][Math.floor(Math.random()*3)]}
- tapOn: { id: "movielist-search-button" }
- tapOn: { id: "search-input" }
- inputText: ${output.termo}
- assertVisible: ${output.termo}
```

Escolhe um título **aleatório** e busca por ele — o mesmo flow testa termos diferentes a cada execução. `${output.termo}` funciona igual a `${SEARCH_TERM}` do `env:`; o JS só entra quando a lógica é complexa demais para um valor fixo.

Recap — o que construímos hoje × o que é com você

Flow	Hoje (ao vivo)	Recurso Maestro
`01-launch`	✅ construído junto	login + `assertVisible`
`02-search`	🏠 você termina em casa (**entrega**)	`env:` + Maestro Studio
`03-favorite`	🏠 você termina em casa (**entrega**)	assert cross-tela
`04-detail`	⭐ opcional (sem nota)	navegação entre telas
`05-js-dynamic`	⭐ opcional (sem nota)	`evalScript`
Jest (Parte A)	🏠 em casa	teste sem emulador

> Você não estava só copiando comando: cada flow carrega um conceito. A entrega são o `02` e o `03`; `04` e `05` repetem o mesmo padrão e são **opcionais** (sem nota).

Parte A — Jest (você termina em casa)

Mesmo app, **testes sem emulador** (feedback em segundos):

- `__tests__/unit/02-authStore.test.ts` — 5 testes (regra: e-mail com `@` + senha de 4+ caracteres)
- `__tests__/integration/01-login-favoritar.test.tsx` — testes 2 e 3 (navegação + estado de verdade)
- O primeiro arquivo de cada pasta vem **resolvido**: copie o jeito dele

```
npm test                    # tudo
npx jest authStore          # só um arquivo
```

> Extensão **Orta.vscode-jest** mostra ✓/✕ ao lado de cada teste.

Atividade — Lab Mobile (10 pts)

Entrega: fork + PR. Prazo: ver Canvas. Enunciado completo no repo.

Critério	Pontos
Testes unitários Jest (`02-authStore` , 5 testes)	2,5
Teste de integração RNTL (testes 2 e 3)	2,5
Flows Maestro `02` e `03` completos (sem TODOs, com asserções)	2
Vídeo rodando em Android Emulator · iOS Simulator (quem não tem Mac: só Android)	0,5 + 0,5
CI verde no fork (Jest no GitHub Actions)	2

> **Eliminatório:** flakiness > 10% em 3 execuções seguidas = −3 pts. Tudo roda **local e de graça**.

Leituras obrigatórias

- **Maestro Docs** (oficial) — *Commands* e *Best practices*
- **Linares-Vásquez et al.** (2017) — *Continuous, Evolutionary and Large-Scale*. ICSME
- **Kochhar et al.** (2015) — *Understanding the Test Automation Culture*. ICST
- **Mobile.dev Blog** — *Why YAML for E2E Tests*

Próximo módulo (Aula 5)

IA Avançada em Testes

- Geração de testes com LLM (grátis, via GitHub Models)
- Self-healing e *healing loop*
- Análise de relatório com IA (inclusive Lighthouse)

Bons estudos!

📧 jackson.96@gmail.com
🔗 linkedin.com/in/3jacksonsmith
🐙 github.com/jacksonsmith